练习2：利用预取优化性能

- **任务**：通过为练习1代码增加数据主动预取机制提升矩阵乘法性能。
- **子任务**：
  - 利用lmbench3获取各级缓存的访问延迟；
  - 利用访问延迟信息为预取指令选择合适的位置。
- **目标**：
  - 学会查阅处理器厂商的软件开发手册选取合适的指令（第一卷10.4.6节数据预取相关内容）；
  - 学会在指令流中为数据预取指令选择合适的位置。

1. 运行 `lmbench3` 工具查看目标处理器信息

注意：需多次运行取均值，单次运行时间约7小时，请合理安排重试次数。

> **说明**
>
> 由于运行耗时较长，课上只需执行相关命令，了解 `lmbench3` 的用法和功能即可，不需等待其运行完毕。运行起来后，可通过快捷键 `^ Ctrl` + `C` 终止运行。

```
cd lab2/tools && tar xf lmbench.tgz && cd lmbench
make results
# 涉及的参数选择
# MULTIPLE COPIES [default 1] 回车
# Job placement selection: 4
# MB [default 10956] 默认值会运行很久，可输入较小值后回车，如8192、4096等
# SUBSET (ALL|HARWARE|OS|DEVELOPMENT) [default all] HARWARE
# FASTMEM [default no] 回车
# SLOWFS [default no] yes
# DISKS [default none] 回车
# REMOTE [default none] 回车
# Processor mhz [default 2688 MHz, 0.3720 nanosec clock] 回车
# FSDIR [default /var/tmp] 回车
# Status output file [default /dev/tty] /tmp/lmbench.test
# Mail results [default yes] no

```

```
# 因为LMBENCH3对新系统的兼容问题，选择完后很快会失败，按下面的步骤继续操作即可
cp -a results ./bin/
make rerun

# 等待运行结束
mv results results.bak
cp -a ./bin/results ./
# 查看延迟信息（如果不够详细可以自行调整make results参数再进行测试）
cd results && make LIST=$(../scripts/os)/*
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/tools/lmbench$ cd results && make LIST=$(../scripts
/os)/*
x86_64-linux-gnu: no data for Memory load parallelism
x86_64-linux-gnu: lat_mem_rand = 114.267

                L M B E N C H  3 . 0   S U M M A R Y
                ------------------------------------
                (Alpha software, do not distribute)

Argument "" isn't numeric in numeric gt (>) at ../scripts/getsummary line 944, <FD> line 1781.

*Local* Communication bandwidths in MB/s - bigger is better
----------------------------------------------------------------------------
Host                 OS  Pipe AF    TCP  File   Mmap  Bcopy  Bcopy  Mem   Mem
                             UNIX      reread reread (libc) (hand) read write
--------- ------------- ---- ---- ---- ------ ------ ------ ------ ---- -----
10.251.13 Linux 5.4.0-2                                 11.9K 9146.8 16.K 12.7K

Memory latencies in nanoseconds - smaller is better
    (WARNING - may not be correct, check graphs)
------------------------------------------------------------------------------
Host                 OS   Mhz   L1 $   L2 $    Main mem    Rand mem    Guesses
--------- -------------   ---   ----   ----    --------    --------    -------
10.251.13 Linux 5.4.0-2  2294 0.9010 2.7240   17.0         114.3
```

在 `lab2/tools` 目录下可以找到“`10.251.137.194.0`”文件，这是 `make results` 指令的输出文件，根据上述的指令将其放入正确位置并执行 `make rerun` 之后的步骤即可。

2. 根据上一步获得的存储信息，为练习1的代码增加数据预取优化（如 `prefetch0` 指令），形成 `src/lab2/gemm_kernel_opt_prefetch.S` 中 `DO_GEMM` 过程（第64行）中的矩阵计算逻辑

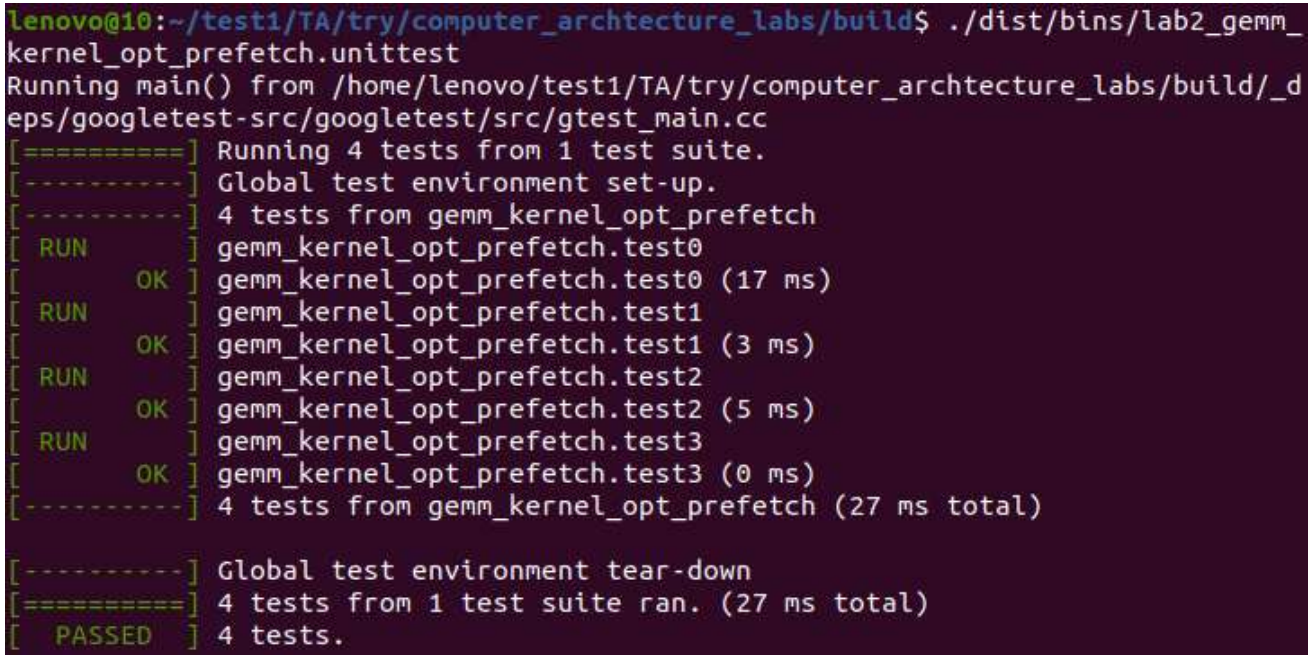
注意：不需自行编写下图中的`DO_GEMM`过程，只需先将`gemm_kernel_baseline.S`中第62~105行`DO_GEMM`的代码替换掉`gemm_kernel_opt_prefetch.S`中的第63~65行，然后在此基础上添加预取优化代码即可。

```
63    .macro DO_GEMM
64        // TODO: 练习2的性能优化任务
65    .endm
```

3. 验证kernel结果的正确性

确保终端当前处于`lab2/build`目录下，然后执行下列命令：

```
cmake -B . -S ../ && cmake --build ./ --target lab2_gemm_kernel_opt_prefetch.unittest
./dist/bins/lab2_gemm_kernel_opt_prefetch.unittest
```



4. 对比优化后的算法与基线的性能（测试用例根据自己的优化选择），根据对比结果分析优化方案的局限性

在`lab2/build`目录下执行下列命令：

```
cmake -B . -S ../ && cmake --build ./ --target lab2_gemm_opt_prefetch
./dist/bins/lab2_gemm_opt_prefetch 1024 128 4
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ ./dist/bins/lab2_gemm_
opt_prefetch 1024 128 4
--- Performance before prefetch optimization ---
GEMM performance info:
                M, K, N: 1024, 128, 4
                Ops: 0.00104858
                Total compute time(s): 0.151045
                Cost(s): 0.000755225
                Benchmark(Gflops): 1.38843
--- Performance for after prefetch optimization ---
GEMM performance info:
                M, K, N: 1024, 128, 4
                Ops: 0.00104858
                Total compute time(s): 0.134581
                Cost(s): 0.000672905
                Benchmark(Gflops): 1.55828
----------------------------
Performance difference(Gflops): 0.169854
```